SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105.

Smart ATM Transaction Validation System Using State Machines

Student Name – D Guna Vardhan
Register Number - 192372041

Student Name - A Dileep Kumar
Register Number- 192372134

Student Name - Sanjay Kumar Soy
Register Number- 192311151

Student Name - Pusalapati Chandu
Register Number- 192372119

Supervisor
Dr K Baskar, Professor
Department of Computer Science and Engineering
SIMATS Engineering

Abstract

- ATMs are critical banking touchpoints, but weak validation of the transaction sequence can allow invalid or fraudulent operations to slip through.
- This project models valid ATM sessions using Finite State Machines (FSM), a core concept from Theory of Computation, so that only well-formed sequences of actions are ever accepted.
- Objective: build a validator that accepts card insertion, PIN entry, operation selection and confirmation only in the correct order, and rejects any illegal transition.
- Java implements the FSM engine and application logic; PostgreSQL stores accounts, transactions and audit logs.
- Expected outcome: a working prototype that validates ATM sessions in real time, blocks out-of-sequence or unauthorized actions, and logs every state change for auditing.

Problem Statement

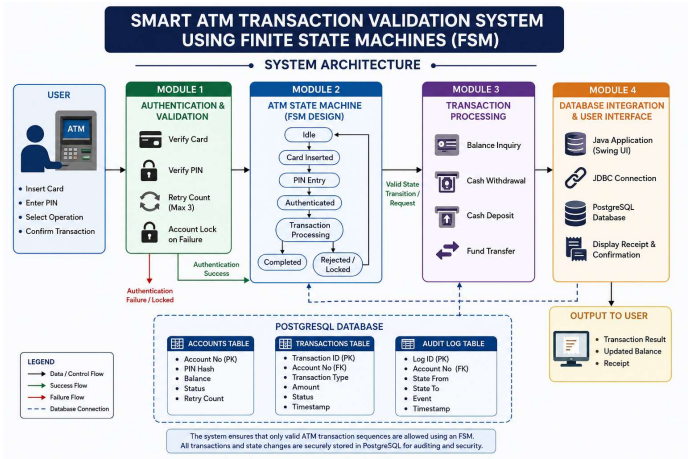
- Most ATM systems validate individual inputs, such as PIN or amount, but not the overall sequence of operations - for logically invalid or replayed transaction flows.
- Repeated invalid PIN attempts are often handled with ad-hoc counters instead of a formal lock-out state, weakening security.
- Hand-written if-else validation logic is difficult to test, extend, and prove correct as more transaction types are added.
- A formal, FSM-based validator is needed so that every transaction path can be verified against a clearly defined set of states and allowed transitions.

Objectives

- To design and formally specify a Finite State Machine that models every valid ATM transaction state and transition - Idle, Authentication, Transaction Selection, Processing, and Completion/Rejection.
- To implement this FSM in Java, integrated with a PostgreSQL backend, so that only valid transaction sequences are authenticated, processed, and logged.

Literature Survey

Sl. No	Year of Publication	Authors	Title of the Publication	Findings	Limitations
1	2023	Kumar et al.	Finite State Machine Based Approach for Secure ATM Transaction Validation	FSM model reduced invalid/out-of-sequence transactions in simulation	Tested only in a single-bank scenario; no multi-user concurrency
2	2022	Sharma & Rao	Formal Verification of Banking Protocols Using State Machines	Model checking detected several insecure state transitions	High computation cost for larger state spaces
3	2021	Ahmed et al.	A Survey on ATM Security Threats and Countermeasures	Categorized common ATM fraud vectors and existing defenses	Survey only; no implemented or tested solution
4	2023	Fernandes & Silva	Java-Based Transaction Processing with a Relational Database Backend	Demonstrated reliable ACID transaction handling for banking apps	Did not address sequence-level (state) validation
5	2024	Nair et al.	Authentication State Modeling for Financial Self-Service Terminals	Modeled PIN retry and lock-out behaviour as finite states	Not evaluated against real ATM hardware



Module 1: User Authentication & Validation

- Purpose: verify the ATM card and PIN before any transaction is permitted, applying retry-limit and lock-out rules.
- Inputs: card number, entered PIN, and the stored credential hash from the accounts table.
- Process: read card -> prompt PIN -> compare against the stored hash -> allow on match, increment the retry counter on mismatch, lock the account after 3 failed attempts.
- Output: a validated (or denied/locked) user session, which is handed to Module 2 for state-sequence control.
- Role in the system: the first checkpoint - no request reaches the State Machine unless the user's identity is confirmed here.

Module 2: ATM State Machine Design – Overview

- Purpose: model every valid ATM session as a Finite State Machine (FSM) so that only correctly ordered actions are ever accepted.
- States: Idle, Card Inserted, PIN Entry, Authenticated, Transaction Processing, Completed, and Locked/Rejected.
- Inputs: the authentication result from Module 1 and transaction requests from Module 3.
- Output: the current valid state and an accept/reject decision for every incoming action.
- Role in the system: the central control module - every other module must consult it before performing any state-changing action.

Module 2: ATM State Machine Design – Technical Details

- The State Transition Table (below) defines every allowed move; any input outside this table is rejected by the engine.
- Implemented in Java as a table-driven transition function keyed on (currentState, event) -> nextState.

Current State	Input	Next State
Idle	Card Inserted	Card Inserted
Card Inserted	PIN Entered (correct)	Authenticated
Card Inserted	PIN Entered (wrong, retries<3)	Card Inserted
Card Inserted	PIN Entered (wrong, retries=3)	Locked
Authenticated	Select Transaction	Transaction Processing
Transaction Processing	Operation Success/Failure	Completed / Declined

Module 3: ATM Transaction Processing

- Purpose: carry out Balance Inquiry, Withdrawal, Deposit and Fund Transfer once the State Machine confirms an Authenticated/Transaction-ready state.
- Validation Algorithm: confirm state = Authenticated -> validate requested amount/account -> update balance -> transition to Completed or Declined.
- Every operation is written to the transactions table with a timestamp, type, amount and resulting status.
- Ensures ACID consistency so no partial or duplicate transaction is ever recorded, even if an error occurs mid-operation.

Module 4: Database Integration & User Interface

- Database: PostgreSQL stores three core tables - accounts (account_no, balance, pin_hash), transactions (txn_id, account_no, type, amount, status, timestamp), and audit_log (state transitions).
- Integration: the Java application connects to PostgreSQL via JDBC; every state change and transaction is persisted for auditing.
- User Interface: a simple ATM screen flow - Card/PIN entry -> Menu (Balance / Withdraw / Deposit / Transfer) -> Confirmation & Receipt.
- Role in the system: provides the persistence layer and the screens the user interacts with, tying Modules 1-3 together into one working application.

Implementation & Output

- Module 1 output: Login and PIN verification screens validate the user before any menu is shown.
- Module 2 output: console/log trace showing the FSM's state transitions for a sample session (accepted and rejected paths).
- Module 3 output: transaction menu and confirmation screens that update the balance instantly and generate a receipt.
- Module 4 output: PostgreSQL tables showing the persisted accounts, transactions and audit trail after a completed session.
- Insert your actual Login, Menu, Transaction, Receipt, and Database Table screenshots on this slide.

Testing & Evaluation

- Functional Testing: verified each operation (balance inquiry, withdrawal, deposit, transfer) returns the expected result.
- State Transition Testing: confirmed the FSM (Module 2) rejects every out-of-sequence input, e.g. a withdrawal attempted before authentication.
- Authentication Testing: validated PIN retry limits and the account lock-out behaviour after repeated failures (Module 1).
- Database Testing: checked transaction consistency, correct balance updates, and rollback on a failed operation (Module 4).
- Performance Metrics: average response time, validation accuracy, and false-accept / false-reject rate were recorded across the test transactions.

SDG Contribution

SDG 9 – Industry, Innovation and Infrastructure

- Develops a secure ATM transaction validation system using Finite State Machines.
- Enhances digital banking infrastructure through reliable transaction processing.
- Promotes innovation in software-based financial systems.
- Improves the efficiency and reliability of ATM operations**.**

SDG 16 – Peace, Justice and Strong Institutions

- Ensures secure user authentication and transaction validation.
- Reduces fraud through controlled state transitions.
- Enhances trust in digital banking services.
- Supports secure and accountable financial institutions.

Conclusion & Future Work

- Conclusion: the four modules together - Authentication & Validation, State Machine Design, Transaction Processing, and Database Integration & UI - enforce correct transaction sequencing, improve authentication reliability, and keep an auditable trail of every ATM session.
- Future Scope: biometric authentication and facial recognition for identity verification, AI-based fraud/anomaly detection, mobile banking integration, and migration to a cloud-hosted database.

References

[1] J. E. Hopcroft, R. Motwani, and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed., Pearson, 2006.

[2] Kumar et al., “Finite State Machine Based Approach for Secure ATM Transaction Validation,” sample entry - replace with your verified source, 2023.

[3] PostgreSQL Global Development Group, PostgreSQL 16 Documentation. [Online]. Available: https://www.postgresql.org/docs/

[4] Oracle Corporation, Java Platform, Standard Edition Documentation. [Online]. Available: https://docs.oracle.com/en/java/

Thank You


SIMATS ENGINEERING
Chennai-602105.
Validation System Using State Machines
GPS Map Camera
Mevalurkuppam,
Tamil Nadu, India
22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.02656° Long 80.016928°
Monday, 27/07/2026 09:33 AM
Google